IF3170 - Inteligensi Artifisial

Tugas Besar



Disusun Oleh Versi3:

Geraldo Artemius - 13524005

Junior Natra Situmorang - 13524055

Reynard Nathanael - 13524103

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

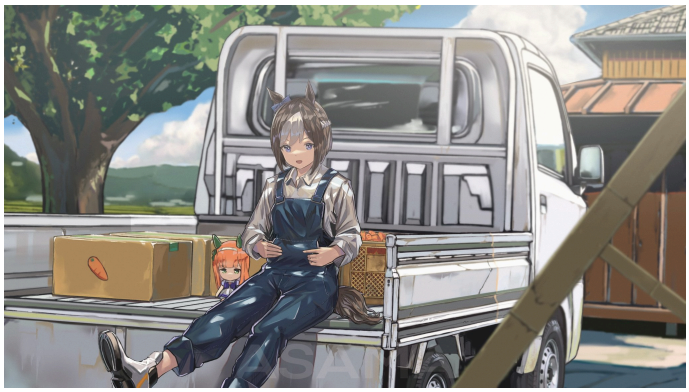
2026

DAFTAR ISI

JUDUL **1**
DAFTAR ISI **2**
BAB I **3**
DESKRIPSI PERSOALAN **3**
BAB II **4**
PEMBAHASAN **4**
2.1 Implementasi Struktur Data dan *Class* **4**
2.2 Implementasi Algoritma Hill-Climbing **15**
2.3 Implementasi Algoritma Simulated Annealing **22**
2.4 Implementasi Algoritma Genetic Algorithm **25**
2.5 Hasil dan Analisis Experiment **31**
2.5.1 Hasil Experiment 31
2.5.2 Analisis Experiment 37
BAB III **41**
KESIMPULAN DAN SARAN **41**
3.1 Kesimpulan 41
3.2 Saran 41
PEMBAGIAN TUGAS **42**
REFERENSI **43**
LAMPIRAN FORM PENGGUNAAN AI **44**

BAB I

DESKRIPSI PERSOALAN



Gambar 1. Special Week, driver truk handal (Sumber: 真夏の銀堡垒🐴C108土曜日 東地区 ホ-10a on X)

Mejiro Express, usaha logistik darat yang dirintis oleh keluarga Mejiro sedang menguji operasionalnya dengan Special Week sebagai pengemudi truk yang dipercaya untuk mengantarkan paket. Namun, dibutuhkan sebuah program yang dapat membantu menentukan susunan paket yang efisien. Persoalan yang dihadapi adalah memilih paket yang akan diangkut serta penempatannya di dalam peti kemas agar menghasilkan total nilai (*value*) sebesar mungkin.

Setiap paket berbentuk kubus atau balok dengan beberapa karakteristik yang menjadi bahan pertimbangan seperti ukuran, berat, nilai, sifat *fragile*, serta perkiraan waktu kedatangan (ETA). Sementara itu, peti kemas juga mempunyai dimensi dan kapasitas berat maksimum. Pemilihan paket harus mempertimbangkan *value*, posisi, orientasi serta kesesuaiannya dengan ruang dan kapasitas yang tersedia.

Paket yang dimuat harus berada sepenuhnya di dalam peti kemas tanpa menembus dinding. Setiap paket harus memiliki satu titik pada alas yang ditopang oleh lantai peti kemas atau bagian atas paket lainnya sehingga tidak tergolong “melayang”. Paket pecah belah (*fragile*) tidak boleh menjadi penopang bagi paket lain, sedangkan total berat paket yang diangkut tidak boleh melebihi kapasitas maksimum truk.

Dalam penyelesaian persoalan tersebut, akan diterapkan algoritma pencarian lokal, yaitu algoritma *Hill-Climbing*, *Simulated Annealing*, dan *Genetic Algorithm*. Berdasarkan definisi *local search*, pencarian akan dimulai dengan konfigurasi acak, kemudian akan mengeksplorasi berbagai calon solusi untuk memperoleh *value* tertinggi. Perubahan yang dilakukan dapat berupa pertukaran antara dua paket, perpindahan posisi, atau rotasi paket. Hasil pencarian selanjutnya ditentukan berdasarkan nilai fungsi objektif dan dilanjutkan dengan logika *searching* masing-masing.

BAB II

PEMBAHASAN

2.1 Implementasi Struktur Data dan *Class*

Source Code:

state.py

```
@dataclass(frozen=True)
class State:
    packages: Tuple[Package, ...]
    truck: Truck

    def __post_init__(self) -> None:
        if not isinstance(self.packages, tuple):
            object.__setattr__(self, "packages",
tuple(self.packages))

        ids = [p.id for p in self.packages]
        if len(ids) != len(set(ids)):
            raise ValueError("ID paket tidak boleh
duplikat.")

    def __len__(self) -> int:
        return len(self.packages)

    def inside(self) -> List[Package]:
        return [p for p in self.packages if p.is_placed]

    def outside(self) -> List[Package]:
        return [p for p in self.packages if not p.is_placed]

    def total_value(self) -> int:
        return sum(p.value for p in self.packages if
p.is_placed)

    def total_weight(self) -> int:
        return sum(p.weight for p in self.packages if
p.is_placed)

    def index_of(self, package_id: str) -> int:
        for i, p in enumerate(self.packages):
```

```
            if p.id == package_id:
                return i

        raise KeyError(f"Paket {package_id!r} tidak
ditemukan.")

    def move(self, i: int, position: Optional[Tuple[int, int,
int]]) -> "State":
        return self._replace_packages({i:
_relocate(self.packages[i], position)})

    def swap(self, i: int, j: int) -> "State":
        if i == j:
            raise ValueError("Tidak bisa menukar paket dengan
dirinya sendiri.")

        a, b = self.packages[i], self.packages[j]

        return self._replace_packages({
            i: _relocate(a, b.position),
            j: _relocate(b, a.position),
        })

    def rotate(self, i: int, axis: str) -> "State":
        return self._replace_packages({i:
self.packages[i].rotate(axis)})

    def _replace_packages(self, changes: Dict[int, Package])
-> "State":
        new_packages = list(self.packages)

        for i, p in changes.items():
            new_packages[i] = p
        return replace(self, packages=tuple(new_packages))

    @classmethod
    def random(
        cls,
        packages: Iterable[Package],
        truck: Truck,
        rng: Optional[random.Random] = None,
        p_inside: float = 0.5,
    ) -> "State":
        from .constraints import is_valid
        rng = rng or random.Random()
        max_attempts = 100 # ganti ganti aja ges

        init_pack =
```

```
tuple(p.remove().get_rotation(rng.randrange(len(Package.ORIEN
TATIONS))) for p in packages)

        state = cls(packages=init_pack, truck=truck)

        order = list(range(len(state.packages)))
        rng.shuffle(order)

        truck_width, truck_length, truck_height =
truck.dimensions

        for i in order:
            if rng.random() >= p_inside:
                continue

            package = state.packages[i]

            if state.total_weight() + package.weight >
truck.max_capacity:
                continue


            fit_rotation = [rotation for rotation in
range(len(Package.ORIENTATIONS)) if
truck.fit(package.get_rotation(rotation).get_dimensions)]

            if not fit_rotation:
                continue

            for _ in range(max_attempts):
                oriented =
package.get_rotation(rng.choice(fit_rotation))

                w, l, h = oriented.get_dimensions

                support_levels = {0}

                for other in state.inside():
                    if not other.is_fragile:

support_levels.add(other.get_bounds()[5])

                valid_levels = sorted(z for z in
support_levels if z + h <= truck_height)

                x = rng.randint(0, truck_width - w)
                y = rng.randint(0, truck_length - l)
                z = rng.choice(valid_levels)
```

```
                cand_package = list(state.packages)
                cand_package[i] = oriented.place(x, y, z)

                candidate = cls(packages=tuple(cand_package),
truck=truck,)

                if is_valid(candidate):
                    state = candidate
                    break
        return state

    def to_dict(self) -> Dict[str, Any]:
        return {
            "truck": self.truck.to_dict(),
            "packages": [p.to_dict() for p in self.packages],
            "total_value": self.total_value(),
            "total_weight": self.total_weight(),
        }

    @classmethod
    def from_dict(cls, data: Dict[str, Any]) -> "State":
        return cls(
            packages=tuple(Package.from_dict(p) for p in
data["packages"]),
            truck=Truck.from_dict(data["truck"]),
        )


def _relocate(package: Package, position: Optional[Tuple[int,
int, int]]) -> Package:
    if position is None:
        return package.remove()
    return package.place(*position)
```

package.py

```
@dataclass(frozen=True)
class Package:
    id: str
    dimensions: Tuple[int, int, int] #(w,l,h)
    value: int
    weight: int
    is_fragile: bool
    eta: int

    position: Optional[Tuple[int, int, int]] = None  #(x,y,z)
    rotation: int = 0
```

```
    ORIENTATIONS: ClassVar[List[Tuple[int, int, int]]] = [
        (0, 1, 2),  #(w,l,h)
        (0, 2, 1),  #(w,h,l)
        (1, 0, 2),  #(l,w,h)
        (1, 2, 0),  #(l,h,w)
        (2, 0, 1),  #(h,w,l)
        (2, 1, 0),  #(h,l,w)
    ]

    ROTATION_SWAP: ClassVar[Dict[str, Tuple[int, int]]] =
{"x": (1, 2), "y": (0, 2), "z": (0, 1)}

    def __post_init__(self) -> None:
        if len(self.dimensions) != 3 or min(self.dimensions)
<= 0:
            raise ValueError(f"Dimensi packet {self.id} harus
3 bilangan positif.")

        if not (0 <= self.rotation < len(self.ORIENTATIONS)):
            raise ValueError("Indeks orientasi harus bernilai
antara 0 dan 5.")

        if self.position is not None and len(self.position)
!= 3:
            raise ValueError("Posisi harus (x,y,z).")

        if type(self.is_fragile) is not bool:
            raise ValueError("is_fragile harus berupa
boolean.")

        if type(self.eta) is not int:
            raise ValueError("ETA harus berupa integer.")

    @property
    def get_dimensions(self) -> Tuple[int, int, int]:
        currentDimension = self.dimensions
        mapping = self.ORIENTATIONS[self.rotation]
        return (currentDimension[mapping[0]],
currentDimension[mapping[1]], currentDimension[mapping[2]])

    def get_bounds(self) -> Optional[Tuple[int, int, int,
int, int, int]]:
        if not self.is_placed:
            return None

        x, y, z = self.position
        w, l, h = self.get_dimensions
```

```
        return (x, y, z, x + w, y + l, z + h)

    def get_rotation(self, idx: int) -> "Package":
        return replace(self, rotation=idx)

    @property
    def is_placed(self) -> bool:
        return self.position is not None

    @property
    def volume(self) -> int:
        w, l, h = self.dimensions
        return w * l * h

    def get_orientation(self) -> Dict[str, str]:
        label = {}
        for axis_idx, dim_idx in
enumerate(self.ORIENTATIONS[self.rotation]):
            label["wlh"[dim_idx]] = "xyz"[axis_idx]

        return {k: label[k] for k in "wlh"}

    def place(self, x: int, y: int, z: int) -> "Package":
        return replace(self, position=(int(x), int(y),
int(z)))

    def remove(self) -> "Package":
        return replace(self, position=None)

    def rotate(self, axis: str) -> "Package":
        if axis not in self.ROTATION_SWAP:
            raise ValueError(f"Sumbu tidak valid: {axis!r}
(pilih 'x', 'y', atau 'z')")

        mapping = list(self.ORIENTATIONS[self.rotation])
        a, b = self.ROTATION_SWAP[axis]
        mapping[a], mapping[b] = mapping[b], mapping[a]
        return replace(self,
rotation=self.ORIENTATIONS.index(tuple(mapping)))

    @classmethod
    def from_dict(cls, data: Dict[str, Any]) -> "Package":
        dimensions = data["dimensions"]
        position = data.get("position")
        return cls(
            id=str(data["id"]),
            dimensions=(int(dimensions["w"]),
int(dimensions["l"]), int(dimensions["h"])),
```

```
            value=data["value"],
            weight=data["weight"],
            is_fragile=bool(data.get("is_fragile",
data.get("isFragile", False))),
            eta=int(data.get("eta", data.get("ETA", 0))),
            position=tuple(position) if position is not None
else None,
            rotation=int(data.get("rotation", 0)),
        )

    def to_dict(self) -> Dict[str, Any]:
        w, l, h = self.dimensions
        return {
            "id": self.id,
            "dimensions": {"w": w, "l": l, "h": h},
            "value": self.value,
            "weight": self.weight,
            "is_fragile": self.is_fragile,
            "eta": self.eta,
            "position": list(self.position) if self.position
else None,
            "rotation": self.rotation,
            "orientation": self.get_orientation(),
        }
```

truck.py

```
@dataclass(frozen=True)
class Truck:
    dimensions: Tuple[int, int, int]
    max_capacity: int

    def __post_init__(self) -> None:
        if len(self.dimensions) != 3 or min(self.dimensions)
<= 0:
            raise ValueError("Dimensi truck harus 3 bilangan
positif.")

        if self.max_capacity < 0:
            raise ValueError("Kapasitas truck harus >= 0.")

    @property
    def volume(self) -> int:
        w, l, h = self.dimensions
        return w * l * h

    def fit(self, size: Tuple[int, int, int]) -> bool:
        w, l, h = size
```

```
        truck_width, truck_length, truck_height =
self.dimensions
        return w <= truck_width and l <= truck_length and h
<= truck_height

    def contain(self, package: Package) -> bool:
        bounds = package.get_bounds()
        if bounds is None:
            return False

        x1, y1, z1, x2, y2, z2 = bounds
        truck_width, truck_length, truck_height =
self.dimensions
        return x1 >= 0 and y1 >= 0 and z1 >= 0 and x2 <=
truck_width and y2 <= truck_length and z2 <= truck_height

    @classmethod
    def from_dict(cls, data: Dict[str, Any]) -> "Truck":
        dimensions = data["dimensions"]
        return cls(
            dimensions=(int(dimensions["w"]),
int(dimensions["l"]), int(dimensions["h"])),
            max_capacity=data.get("max_capacity",
data.get("maxCapacity")),
        )

    def to_dict(self) -> Dict[str, Any]:
        w, l, h = self.dimensions
        return {
            "dimensions": {"w": w, "l": l, "h": h},
            "max_capacity": self.max_capacity,
        }
```

Deskripsi:

No	Nama Variable	Type (Struktur Data / *Class* / Method/ Atribut)	Deskripsi
1	Package	*Class*	Merepresentasikan sebuah paket yang berbentuk kubus atau balok. Class ini menyimpan identitas (ID), nilai, berat, status *fragile*, ETA, posisi, dan orientasi paket.
2	Truck	*Class*	Merepresentasikan sebuah truk lebih

			tepatnya peti kemas yang menjadi tempat penyusunan paket. *Class* ini menyimpan dimensi ruang dan kapasitas maksimum berat, serta menyediakan operasi-operasi pembantu.
3	State	*Class*	Merepresentasikan konfigurasi lengkap sebuah permasalahan, seperti seluruh paket beserta posisi dan orientasinya. *State* akan mencakup paket yang di dalam dan di luar truk. Algoritma *local search* yang nantinya mengevaluasi *class* ini serta pembuatan *state* baru.
4	Package.id	Atribut	Menyimpan identitas unik untuk sebuah paket.
5	Package.dimensions	Struktur Data	Menyimpan dimensi paket dalam urutan (w, l, h), yaitu lebar, panjang, dan tinggi paket.
6	Package.value	Atribut	Menyimpan nilai (*value*) suatu paket. Nilai paket akan dijadikan sebagai pertimbangan untuk perhitungan *objective value*.
7	Package.weight	Atribut	Menyimpan berat dari suatu paket. Atribut ini penting untuk menjaga konsistensi persyaratan penampungan dalam truk.
8	Package.is_fragile	Atribut	Menandai apakah sebuah paket berisi barang pecah belah. Atribut ini penting untuk menjaga konsistensi bahwa paket yang *fragile* tidak boleh menjadi penopang.
9	Package.eta	Atribut	Menyimpan perkiraan waktu kedatangan sebuah paket dalam hari.
10	Package.position	Struktur Data	Menyimpan koordinat sudut minimum paket dalam bentuk (x, y, z). Optional karena paket dapat berada di luar truk sehingga lebih cocok dengan nilai `None`.
11	Package.rotation	Atribut	Menyimpan indeks orientasi paket yang sesuai pada *list* `ORIENTATIONS` dengan rentang 0-5.
12	Package.ORIENTATIONS	Struktur Data	Menyimpan enam pemetaan dimensi paket ke sumbu koordinat. Contohnya, (0, 1, 2)

			akan menghasilkan (w, l, h), sedangkan (0, 2, 1) akan menghasilkan (w, h, l).
13	Package.ROTATION_SWAP	Struktur Data	Memetakan sumbu rotasi ke pasangan indeks dimensi yang ditukar. Rotasi terhadap X menukar indeks 1 dan 2, terhadap Y akan menukar indeks 0 dan 2, serta terhadap Z menukar indeks 0 dan 1.
14	Truck.dimensions	Struktur Data	Ukuran ruang peti kemas dalam urutan (w, l, h). Nilainya akan menjadi batas penempatan paket pada sumbu X, Y, dan Z.
15	Truck.max_capacity	Atribut	Menyimpan nilai kapasitas maksimum berat paket yang bisa ditampung di truk.
16	State.packages	Struktur Data	Menyimpan seluruh paket yang ada dalam suatu konfigurasi. Status paket ada di dalam atau di luar truk ditentukan dari nilai atribut `position`.
17	State.truck	Atribut	Menyimpan truk yang digunakan pada konfigurasi tersebut.
18	Package.get_dimensions	Method	Menghasilkan dimensi paket sesuai orientasi saat ini dengan mengambil pemetaan dari `ORIENTATIONS[rotation]`.
19	Package.get_bounds	Method	Menghasilkan batas ruang paket dalam bentuk (x1, y1, z1, x2, y2, z2). Tiga komponen pertama merupakan koordinat minimum, sedangkan tiga komponen terakhir diperoleh dengan menambahkan dimensi sesuai orientasi.
20	Package.get_rotation	Method	Membuat salinan paket dengan indeks orientasi yang ditentukan oleh `idx`.
21	Package.is_placed	Method	Berisi nilai *boolean* yang akan mengembalikan `True` jika position tidak bernilai `None`.
22	Package.volume	Method	Menghitung volume paket melalui perkalian dimensi dasar.
23	Package.get_orienta	Method	Menghasilkan pemetaan dimensi dasar w, l,

	tion		dan h terhadap sumbu x, y, dan z.
24	Package.place	Method	Membuat salinan paket dengan posisi baru.
25	Package.remove	Method	Membuat salinan paket dengan position bernilai None untuk merepresentasikan paket di luar truk.
26	Package.rotate	Method	Menghasilkan paket dengan orientasi setelah rotasi 90° terhadap sumbu yang dipilih.
27	Package.from_dict	Method	Membentuk objek Package dari *dictionary*, misalnya seperti hasil pembacaan `.JSON`.
28	Package.to_dict	Method	Mengubah objek paket menjadi *dictionary* yang berisi ID, dimensi dasar, nilai, berat, status *fragile*, ETA, posisi, indeks rotasi, dan pemetaan orientasi.
29	Truck.volume	Method	Menghitung volume ruang peti kemas melalui perkalian lebar, panjang, dan tinggi truk biasa.
30	Truck.fit	Method	Memeriksa apakah suatu ukuran paket dapat muat berdasarkan perbandingan setiap dimensinya dengan dimensi truk.
31	Truck.contain	Method	Memeriksa apakah sebuah paket berada di dalam peti kemas truk.
32	Truck.from_dict	Method	Membentuk objek Truck dari *dictionary*.
33	Truck.to_dict	Method	Mengubah informasi truk menjadi *dictionary* yang memuat dimensi dan kapasitas maksimum.
34	State.inside	Method	Menghasilkan daftar paket yang memiliki posisi penempatan melalui pemeriksaan `is_placed`.
35	State.outside	Method	Kebalikan dari *method* inside, mengembalikan daftar paket yang tidak memiliki posisi penempatan atau position bernilai `None`.
36	State.total_value	Method	Menjumlahkan nilai seluruh paket yang berada di dalam truk.

37	State.total_weight	Method	Menjumlahkan berat seluruh paket yang memiliki posisi penempatan atau di dalam truk.
38	State.index_of	Method	Mencari indeks paket berdasarkan ID dalam *tuple* packages.
39	State.move	Method	Membentuk konfigurasi baru dengan memindahkan paket pada indeks `i` ke posisi yang diberikan.
40	State.swap	Method	Menukar posisi dua paket, termasuk ketika salah satunya berada di luar truk.
41	State.rotate	Method	Membentuk konfigurasi baru dengan merotasi paket pada indeks `i` terhadap sumbu tertentu.
42	State._replace_packages	Method	Menerima *dictionary* yang memetakan indeks paket ke objek paket pengganti.
43	State.random	Method	Membentuk konfigurasi awal secara acak. Proses dimulai dengan seluruh paket di luar truk dan orientasi acak, kemudian urutan pemrosesan paket diacak.
44	State.from_dict	Method	Membentuk objek *state* dari *dictionary* yang memuat data truk dan daftar paket.
45	State.to_dict	Method	Mengubah konfigurasi menjadi *dictionary* yang memuat data truk, seluruh paket, total nilai muatan, dan total berat muatan.

2.2 Implementasi Algoritma Hill-Climbing

Source Code:

Hill-Climbing Steepest Ascent

```
def hc_steepest_accent(initial_state: State, max_iterations:
int = 1000, check_fragile: bool = True) -> Dict[str, Any]:
    validate_input(initial_state, max_iterations,
check_fragile)
    start_time = time.perf_counter()
    current_state = initial_state
    current_value = objective(current_state,num, check_fragile)
    obj_values = [current_value]
```

```
    for _ in range(max_iterations):
        neighbor = best_neighbor(current_state, check_fragile)
        if neighbor is None:
            break

        next_state, next_value = neighbor
        if next_value <= current_value:
            break

        current_state, current_value = next_state, next_value
        obj_values.append(current_value)

    end_time = time.perf_counter()
    duration = end_time - start_time

    return {
        "algorithm": "hc_steepest_accent",
        "initial_state": initial_state,
        "initial_value": obj_values[0],
        "final_state": current_state,
        "final_value": current_value,
        "objective_values": obj_values,
        "iterations": len(obj_values) - 1,
        "max_iterations": max_iterations,
        "duration": duration
    }
```

Hill-Climbing with Sideways Move

```
def hc_sideways_move(initial_state: State, max_sideways:
int = 100, max_iterations: int = 1000, check_fragile: bool
= True) -> Dict[str, Any]:
    validate_input(initial_state, max_iterations,
check_fragile)
    if type(max_sideways) is not int or max_sideways < 0:
        raise ValueError("max_sideways harus berupa
integer >= 0.")

    start_time = time.perf_counter()
    rng = random.Random()
    cur_state = initial_state
    cur_val = objective(cur_state, num, check_fragile)
    obj_values = [cur_val]
    sideways_count = 0
```

```
    for _ in range(max_iterations):
        neighbor = best_neighbor(cur_state, check_fragile,
rng)
        if neighbor is None:
            break

        next_state, next_value = neighbor
        if next_value < cur_val:
            break

        if next_value == cur_val:
            if sideways_count >= max_sideways:
                break
            sideways_count += 1

        cur_state, cur_val = next_state, next_value
        obj_values.append(cur_val)

        if max_sideways > 0 and sideways_count ==
max_sideways:
            break

    duration = time.perf_counter() - start_time

    return {
        "algorithm": "hc_sideways_move",
        "initial_state": initial_state,
        "initial_value": obj_values[0],
        "final_state": cur_state,
        "final_value": cur_val,
        "objective_values": obj_values,
        "iterations": len(obj_values) - 1,
        "max_iterations": max_iterations,
        "max_sideways": max_sideways,
        "sideways_count": sideways_count,
        "duration": duration
    }
```

Stochastic Hill-Climbing

```
def hc_stochastic(initial_state: State, max_iterations:
int = 1000, check_fragile: bool = True) -> Dict[str, Any]:
    validate_input(initial_state, max_iterations,
```

```
check_fragile)
    start_time = time.perf_counter()
    rng = random.Random()
    cur_state = initial_state
    cur_val = objective(cur_state, check_fragile)
    obj_values = [cur_val]

    for _ in range(max_iterations):
        if cur_state.packages:
            neighbor = get_random_neighbor(cur_state, rng,
check_fragile=check_fragile)
        else:
            neighbor = None

        if neighbor is not None:
            _, next_state = neighbor
            next_value = objective(next_state, num,
check_fragile)
            if next_value > cur_val:
                cur_state, cur_val = next_state,
next_value

        obj_values.append(cur_val)

    duration = time.perf_counter() - start_time

    return {
        "algorithm": "hc_stochastic",
        "initial_state": initial_state,
        "initial_value": obj_values[0],
        "final_state": cur_state,
        "final_value": cur_val,
        "objective_values": obj_values,
        "iterations": len(obj_values) - 1,
        "max_iterations": max_iterations,
        "duration": duration
    }
```

Random Restart Hill-Climbing

```
def hc_random_restart(initial_state: State, num: int,
max_restarts: int = 10, max_iterations: int = 1000,
check_fragile: bool = True) -> Dict[str, Any]:
```

```
    validate_input(initial_state, max_iterations,
check_fragile)
    if type(max_restarts) is not int or max_restarts < 0:
        raise ValueError("max_restarts harus integer >=
0.")

    start_time = time.perf_counter()
    cur_state = initial_state
    initial_value = objective(initial_state, num,
check_fragile)
    best_state = initial_state
    best_val = initial_value
    restart_count = 0
    iterations_in_restarts = []
    obj_values = []

    for restart in range(max_restarts + 1):
        if restart > 0:
            cur_state =
State.random(initial_state.packages, initial_state.truck)
            restart_count += 1

        result = hc_steepest_accent(cur_state,
max_iterations, check_fragile) # seinget gw random restart
itu pake SA dl?

iterations_in_restarts.append(result["iterations"])
        obj_values.append(result["objective_values"])

        if result["final_value"] > best_val:
            best_state = result["final_state"]
            best_val = result["final_value"]

    duration = time.perf_counter() - start_time

    return {
        "algorithm": "hc_random_restart",
        "initial_state": initial_state,
        "initial_value": initial_value,
        "final_state": best_state,
        "final_value": best_val,
        "objective_values": obj_values,
        "iterations": sum(iterations_in_restarts),
        "iterations_in_restarts": iterations_in_restarts,
```

```
        "restart_count": restart_count,
        "max_restarts": max_restarts,
        "max_iterations": max_iterations,
        "duration": duration
    }
```

Deskripsi:

No	Nama Fungsi	Type (Function/Procedure)	Deskripsi
1	validate_input	Procedure	Memeriksa kesesuaian input sebelum pencarian dimulai. Pemeriksaan seperti `max_iterations` yang harus berupa integer tidak negatif dan konfigurasi awal yang harus memenuhi *constraint* melalui pemanggilan `is_valid()`. Parameter `check_fragile` akan dipakai dalam pemeriksaan aturan paket pecah belah. Jika input tidak memenuhi ketentuan, prosedur akan berhenti.
2	best_neighbor	Function	Menentukan *neighbor* dengan nilai *objective* tertinggi dari daftar yang dihasilkan oleh get_all_neighbor(). Fungsi menghitung *objective* setiap *candidate*, mencari nilai maksimum, kemudian mengumpulkan seluruh *candidate* dengan nilai tersebut. Jika objek pembangkit angka acak rng diberikan, satu *candidate* terbaik dipilih secara acak; jika tidak, *candidate* terbaik pertama dipilih. Fungsi mengembalikan pasangan *state* terpilih dan nilai *objective*-nya, atau None jika tidak tersedia neighbor. Keputusan menerima perpindahan dilakukan oleh fungsi algoritma yang memanggilnya.
Hill-Climbing Steepest Ascent			
3	hc_steepest_accent	Function	Melakukan pencarian dengan memilih *neighbor* yang bernilai *objective* tertinggi melalui `best_neighbor()`. Aktivitas

			hanya dapat diterima apabila nilai *neighbor* lebih tinggi daripada nilai *state* saat ini. Setelah perpindahan diterima, *state* dan nilai *objective* diperbarui. Pencarian berhenti apabila tidak tersedia *neighbor*, *neighbor* terbaik memiliki nilai lebih kecil atau sama dengan *state* saat ini, atau batas max_iterations tercapai.
Hill-Climbing with Sideways Move			
4	hc_sideways_move	Function	Melakukan pencarian dengan memilih *neighbor* terbaik dan mengizinkan perpindahan ke *neighbor* yang memiliki nilai *objective* sama dengan *state* saat ini. Apabila beberapa *neighbor* memiliki nilai terbaik yang sama, pemilihan dilakukan secara acak melalui `best_neighbor()`. Perpindahan dengan nilai sama yang akan menambah `sideways_count`, sedangkan perpindahan dengan nilai lebih tinggi memperbarui *state* tanpa menambah penghitung tersebut. Pencarian berhenti apabila tidak tersedia *neighbor*, *neighbor* terbaik bernilai lebih rendah, batas total sideways tercapai, atau batas iterasi tercapai.
Stochastic Hill-Climbing			
5	hc_stochastic	Function	Melakukan pencarian dengan mencoba satu *neighbor* acak melalui `get_random_neighbor()` pada setiap iterasi. Apabila *neighbor* diperoleh dan nilai *objective*-nya lebih tinggi daripada *state* saat ini, maka perpindahan diterima. *Neighbor* yang bernilai sama atau lebih rendah ditolak sehingga *state* akan tetap. Jika tidak diperoleh *neighbor*, maka *state* juga tetap dan pencarian dilanjutkan. Pencarian akan berlangsung sebanyak `max_iterations`.
Random Restart Hill-Climbing			

6	hc_random_restart	Function	Menjalankan `hc_steepest_accent()` dari konfigurasi awal, kemudian mengulangi pencarian dengan konfigurasi acak baru melalui `State.random()`. Setelah setiap pencarian selesai, nilai akhirnya dibandingkan dengan nilai terbaik yang telah ditemukan. *State* terbaik diperbarui hanya jika hasil pencarian tersebut lebih tinggi. Proses dilakukan hingga jumlah restart mencapai `max_restarts` dan pencarian terakhir selesai, dengan `max_iterations`.

2.3 Implementasi Algoritma Simulated Annealing

Source Code:

```
import math
import random
import time
from typing import Any, Dict, Iterable, List, Optional

from problem import Package, State, Truck, get_random_neighbor,
objective

def exponential_cooling(T_start, alpha, iteration):
    return T_start * (alpha ** iteration)

def validate_parameters(
    initial_temperature: float,
    cooling_rate: float,
    minimum_temperature: float,
    max_iterations: int,
) -> None:
    if initial_temperature <= 0:
        raise ValueError("initial_temperature harus lebih besar
dari 0.")
    if minimum_temperature < 0:
        raise ValueError("minimum_temperature tidak boleh
negatif.")
    if initial_temperature <= minimum_temperature:
        raise ValueError(
            "initial_temperature harus lebih besar dari
minimum_temperature."
        )
```

```
    if not 0 < cooling_rate < 1:
        raise ValueError("cooling_rate harus berada di antara 0
dan 1.")
    if max_iterations < 1:
        raise ValueError("max_iterations harus minimal 1.")


def acceptance_probability(
    current_value: int, candidate_value: int, temperature:
float
) -> float:
    if temperature <= 0:
        raise ValueError("temperature harus lebih besar dari
0.")
    if candidate_value >= current_value:
        return 1.0
    return math.exp((candidate_value - current_value) /
temperature)


def simulated_annealing(
    initial_state: State,
    num: int,
    initial_temperature: float = 100.0,
    cooling_rate: float = 0.995,
    minimum_temperature: float = 0.1,
    max_iterations: int = 10000,
    seed: Optional[int] = None,
    check_fragile: bool = True,
    verbose: bool = True,
    log_every: int = 100,
) -> Dict[str, Any]:
    validate_parameters(
        initial_temperature,
        cooling_rate,
        minimum_temperature,
        max_iterations,
    )
    if log_every < 1:
        raise ValueError("log_every harus minimal 1.")

    rng = random.Random(seed)
    package_list = tuple(initial_state.packages)
    start_time = time.perf_counter()

    initial_value = objective(initial_state, num,
check_fragile)
    current_state = initial_state
```

```
    current_value = initial_value
    objective_history: List[int] = [current_value]
    acceptance_factor_history: List[float] = [1.0]
    stuck_frequency = 0

    temperature = initial_temperature
    iteration = 0

    while (
        package_list
        and temperature > minimum_temperature
        and iteration < max_iterations
    ):
        if iteration >= max_iterations:
            break

        iteration += 1
        acceptance_factor = 1.0
        neighbor = (
            get_random_neighbor(current_state, rng,
check_fragile=check_fragile)
            if package_list
            else None
        )

        if neighbor is not None:
            _, candidate_state = neighbor
            candidate_value = objective(candidate_state, num,
check_fragile)
            probability = acceptance_probability(
                current_value, candidate_value, temperature
            )
            acceptance_factor = probability

            if rng.random() < probability: #rng.random() bisa
diubah jadi static value sesuai kebutuhan
                current_state = candidate_state
                current_value = candidate_value

        is_stuck = current_value == objective_history[-1]
        if is_stuck:
            stuck_frequency += 1

        objective_history.append(current_value)
        acceptance_factor_history.append(acceptance_factor)

        if verbose and iteration % log_every == 0:
            print(
```

```
                f"[Simulated Annealing] Iterasi {iteration:5d}
| "
                f"suhu = {temperature:.4f} | "
                f"nilai saat ini = {current_value}"
            )

        temperature = exponential_cooling(
            initial_temperature, cooling_rate, iteration
        )

    duration = time.perf_counter() - start_time

    return {
        "algorithm": "simulated_annealing",
        "seed": seed,
        "initial_state": initial_state,
        "initial_value": initial_value,
        "final_state": current_state,
        "final_value": current_value,
        "objective_history": objective_history,
        "acceptance_history": acceptance_factor_history,
        "stuck_frequency": stuck_frequency,
        "duration": duration,
    }
```

Deskripsi:

No	Nama Fungsi	Type (Function/Procedure)	Deskripsi
1	Simulated Annealing	Function	Fungsi utama untuk melakukan *local search* Simulated Annealing.
2	Acceptance_Probability	Function	Fungsi yang melakukan perhitungan nilai plot $e^{E/T}$.
3	Validate_Parameter	Function	Fungsi yang melakukan validasi nilai parameter fungsi utama.
4	exponential_cooling	Function	Fungsi untuk menurunkan nilai Temperature secara eksponensial.

2.4 Implementasi Algoritma Genetic Algorithm

Source Code:

```
MUTATION_PROB = 0.3

def initial_population(packages, truck, population_size: int,
rng: random.Random) -> List[State]:
    return [State.random(packages, truck, rng) for _ in
range(population_size)]

def fitness_function(individual: State, num: int) -> int:
    return objective(individual, num)

def fitness_percentage(fitness_values: List[int]) ->
List[float]:
    total_fitness = sum(fitness_values)

    if total_fitness == 0:
        return [1 / len(fitness_values)] * len(fitness_values)

    return [fitness / total_fitness for fitness in
fitness_values]

def roulette_wheel(probabilities: List[float], rng:
random.Random) -> int:
    roulette_result = rng.random()

    cumulative = 0.0
    for index, percentage in enumerate(probabilities):
        cumulative += percentage

        if roulette_result <= cumulative:
            return index

    return len(probabilities) - 1

def selection(population: List[State], probabilities:
List[float], rng: random.Random) -> List[Tuple[State, State]]:
    total_pairs = (len(population) + 1) // 2

    selected_parents = []
    for _ in range(total_pairs):
        parent1 = population[roulette_wheel(probabilities,
rng)]
        parent2 = population[roulette_wheel(probabilities,
rng)]
        selected_parents.append((parent1, parent2))

    return selected_parents

def crossover(parent1: State, parent2: State, rng:
```

```
random.Random) -> Tuple[State, State]:
    total_genes = len(parent1.packages)

    if total_genes < 2:
        return parent1, parent2

    crossover_point = rng.randint(1, total_genes - 1)

    genes_child1 = parent1.packages[:crossover_point] +
parent2.packages[crossover_point:]
    genes_child2 = parent2.packages[:crossover_point] +
parent1.packages[crossover_point:]

    child1 = repair(genes_child1, genes_child2, parent1)
    child2 = repair(genes_child2, genes_child1, parent1)

    return child1, child2

def repair(genes, backup_genes, reference: State) -> State:
    def base_height(package) -> float: #nilai z paket
        return package.position[2] if package.is_placed else
float("inf")

    insert_order = sorted(range(len(genes)), key=lambda i:
base_height(genes[i]))

    child = State(tuple(p.remove() for p in
reference.packages), reference.truck)

    for i in insert_order:
        for gene in (genes[i], backup_genes[i]):
            if not gene.is_placed:
                continue

            trial_state = child._replace_packages({i: gene})

            if is_valid(trial_state):
                child = trial_state
                break

    return child

def mutation(child: State, rng: random.Random, mutation_prob:
float = MUTATION_PROB) -> State:
    if rng.random() < mutation_prob:
        result = get_random_neighbor(child, rng)

        if result is not None:
```

```
            _, mutated_child = result
            return mutated_child

    return child

def genetic_algorithm(
    packages,
    truck,
    num: int,
    population_size: int = 30, #bebas
    generations: int = 50, #bebas
    seed: Optional[int] = None,
    verbose: bool = True,
    log_every: int = 10,
) -> Dict[str, Any]:
    rng = random.Random(seed)
    start_time = time.perf_counter()

    population = initial_population(packages, truck,
population_size, rng)

    initial_best = max(population, key=lambda x:
fitness_function(x, num))
    best_state = initial_best
    best_value = fitness_function(initial_best, num)

    best_objective: List[int] = []
    mean_objective: List[float] = []

    for iteration in range(generations + 1):
        fitness_values = [fitness_function(individual, num) for
individual in population]
        probabilities = fitness_percentage(fitness_values)

        current_best_index = max(range(population_size),
key=lambda i: fitness_values[i])
        current_best_value = fitness_values[current_best_index]
        current_mean_value = sum(fitness_values) /
population_size

        best_objective.append(current_best_value)
        mean_objective.append(current_mean_value)

        if current_best_value > best_value:
            best_state, best_value =
population[current_best_index], current_best_value

        if verbose and iteration % log_every == 0:
```

```
            print(f"[Genetic Algorithm] Generasi {iteration:3d}
| max = {current_best_value} | rata-rata =
{current_mean_value:.1f}")

        if iteration == generations:
            break

        selected_parents = selection(population, probabilities,
rng)

        new_population: List[State] = []
        for parent1, parent2 in selected_parents:
            child1, child2 = crossover(parent1, parent2, rng)

            new_population.append(mutation(child1, rng))
            new_population.append(mutation(child2, rng))

        population = new_population[:population_size]

    duration = time.perf_counter() - start_time

    return {
        "algorithm": "genetic_algorithm",
        "population_size": population_size,
        "generations": generations,
        "mutation_prob": MUTATION_PROB,
        "seed": seed,
        "initial_state": initial_best,
        "initial_value": fitness_function(initial_best, num),
        "final_state": best_state,
        "final_value": best_value,
        "best_objective": best_objective,
        "mean_objective": mean_objective,
        "duration": duration,
    }
```

Deskripsi:

No	Nama Fungsi	Type (Function/Procedure)	Deskripsi
1	initial_population	Function	Membuat populasi awal sesuai dengan *population_size*. Setiap individu yang dibuat adalah *state* yang dibuat secara *random* namun valid.

2	fitness_function	Function	Menghitung nilai *fitness* dari sebuah individu menggunakan *objective function* yang ditetapkan.
3	fitness_percentage	Function	Menghitung persentase *fitness* dari setiap individu dibagi dengan total *fitness* seluruh populasi. Kalau total *fitness* = 0, maka setiap individu memiliki peluang yang sama.
4	roulette_wheel	Function	Memilih satu individu dengan menggunakan *roulette wheel*. Fungsi ini akan menghasilkan nilai acak antara 0 hingga 1, lalu menjumlahkan persentase dari *fitness* percentage untuk mengembalikan indeks individu yang terpilih.
5	selection	Function	Merupakan tahap seleksi dengan memilih *parent1* dan *parent2* menggunakan *roulette_wheel*. Dengan jumlah pasangan pasti setengah dari ukuran populasi.
6	crossover	Function	Melakukan *single-point crossover* dengan memilih satu titik potong secara acak, kemudian menukar gen dari kedua orang tua sehingga terbentuk dua *child.*
7	repair	Function	Memperbaiki hasil *crossover* agar tidak melanggar *constraints*. Jadi *package* akan ditaruh satu per satu dari posisi z terendah, kemudian jika gen utama tidak valid akan dicoba gen kedua dari orang tua lainnya, dan jika keduanya tidak valid maka *package* akan ditaruh di luar *truck*.
8	base_height	Function	Fungsi di dalam fungsi *repair.* Fungsi ini akan memberikan nilai z dari sebuah *package. Package* yang berada di luar *truck* diberi nilai tak hingga.
9	mutation	Function	Melakukan mutasi pada hasil *crossover* yang valid menggunakan nilai *mutation_prob*. Jika terjadi mutasi, maka *child* akan diganti dengan satu *neighbor*

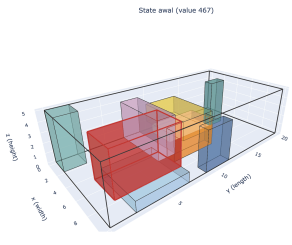
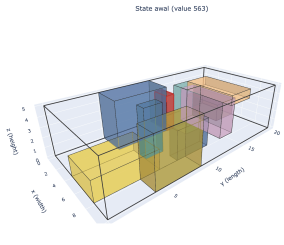
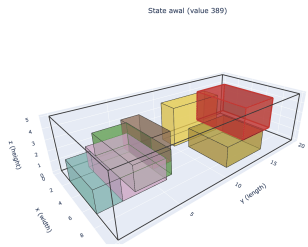
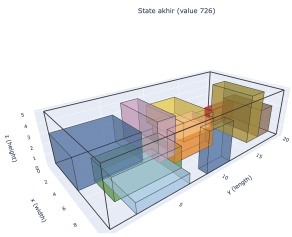
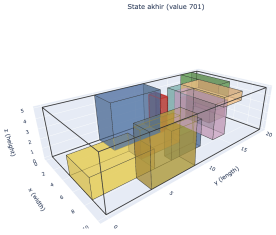
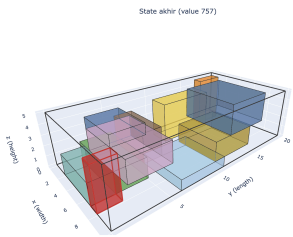
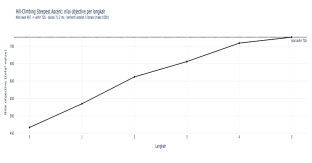
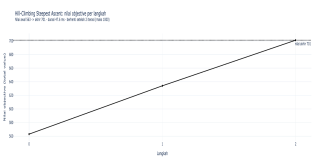
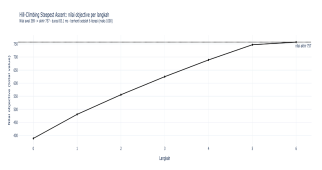
			dengan langkah yang acak.
10	genetic_algorithm	Function	Fungsi utama *genetic algorithm* yang mengulangi tahap *fitness, selection, crossover, mutation* sebanyak generations yang di-*input*.

2.5 Hasil dan Analisis Experiment

2.5.1 Hasil Experiment

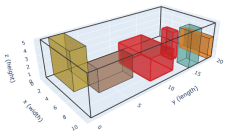
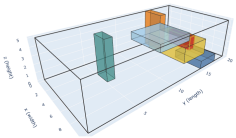
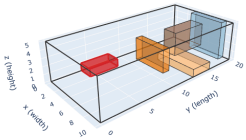
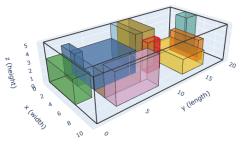
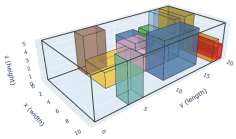
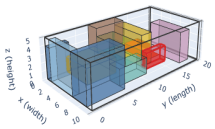
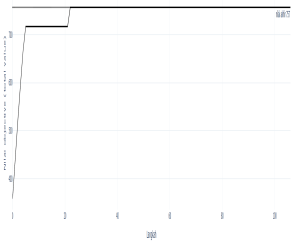
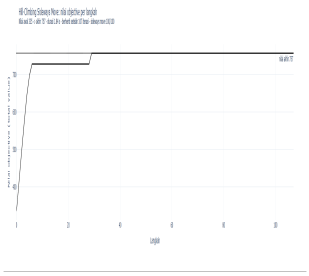
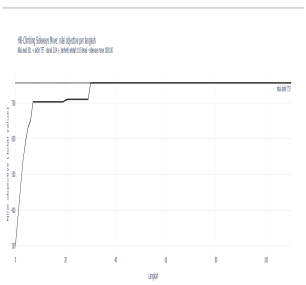
Hasil experiment dari setiap algoritma dapat dilihat di bawah ini:

Hill-Climbing Steepest Ascent

Percobaan ke-i	**1**	**2**	**3**
State Awal			
State Akhir			
Nilai *Objective* Function akhir	726	701	757
Plot nilai *objective* terhadap banyak iterasi			
Durasi Pencarian	71.2 ms	47.6 ms	83.1 ms
Banyak iterasi	5	2	6

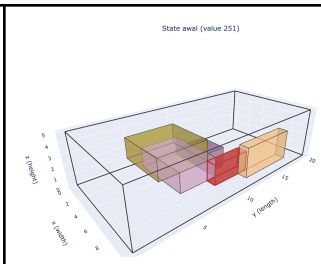
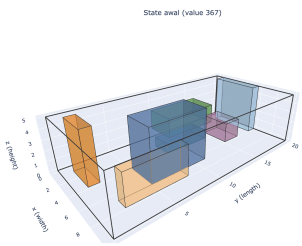
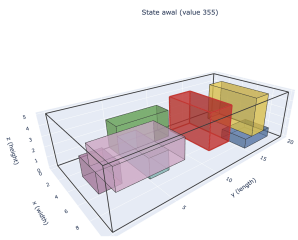
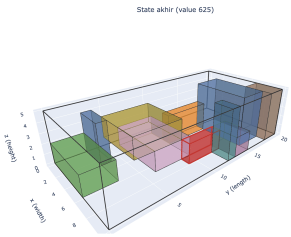
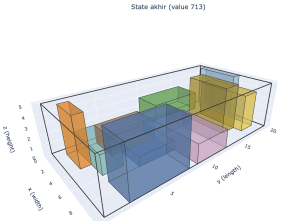
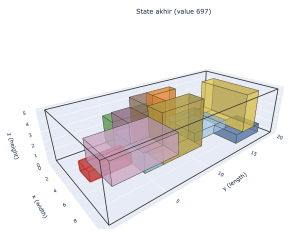
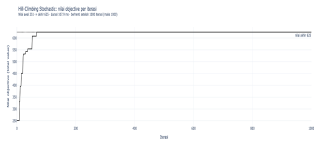
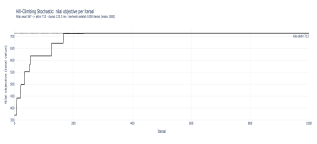
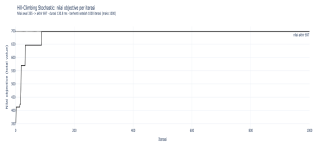
hingga proses berhenti			

Hill-Climbing Sideways Move

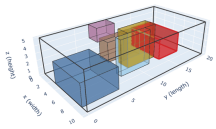
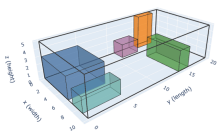
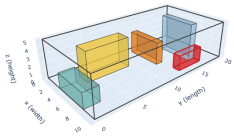
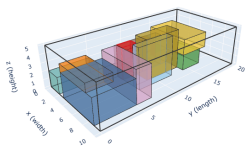
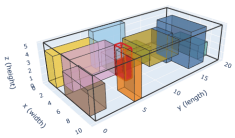
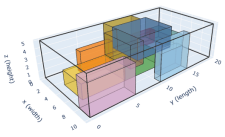
Percobaan ke-i	**1**	**2**	**3**
State Awal			
State Akhir			
Nilai *Objective* Function akhir	757	757	757
Plot nilai *objective* terhadap banyak iterasi			
Durasi Pencarian	1.68 s	1.84 s	2.04 s
Banyak iterasi hingga proses pencarian berhenti	106 Iterasi	107 Iterasi	110 Iterasi
maximum sideways move	100/100	100/100	100/100

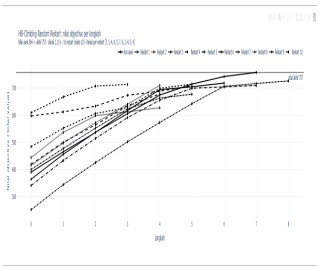
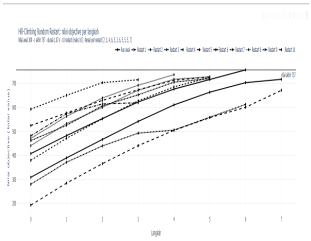
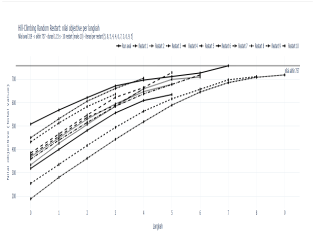
Hill-Climbing Stochastic

Percobaan ke-i	**1**	**2**	**3**

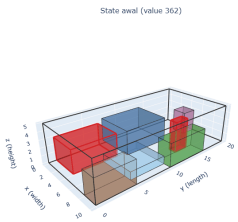
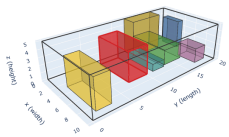
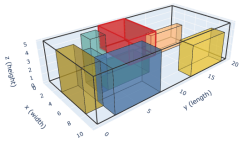
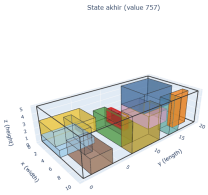
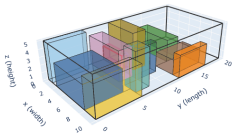
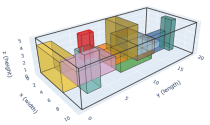
State Awal			
State Akhir			
Nilai *Objective Function* akhir	625	713	697
Plot nilai *objective* terhadap banyak iterasi			
Durasi Pencarian	107.9 ms	123.3 ms	130.8 ms
Banyak iterasi hingga proses berhenti	1000 (*default*)	1000 (*default*)	1000 (*default*)

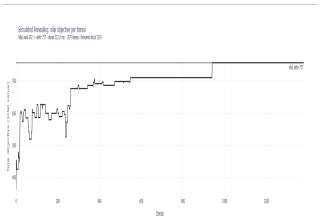
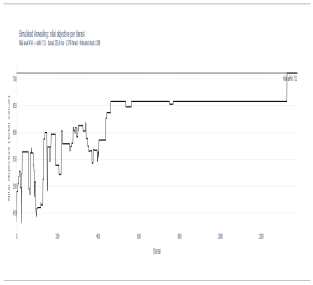
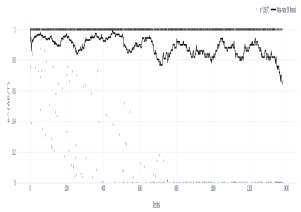
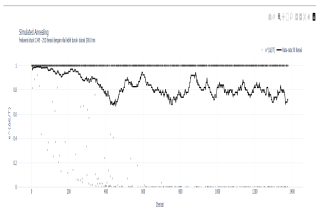
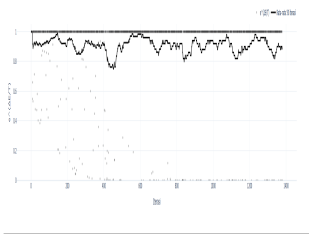
Hill-Climbing Random Restart

Percobaan ke-i	**1**	**2**	**3**
State Awal			
State Akhir			

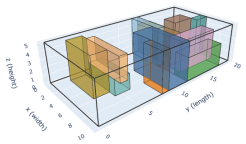
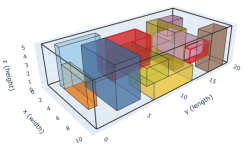
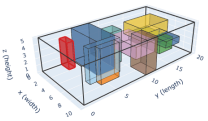
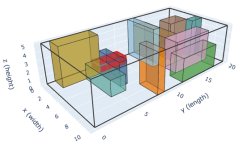
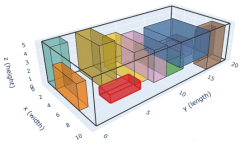
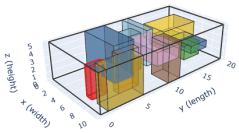
Nilai *Objective* Function akhir	757	757	757
Plot nilai *objective* terhadap banyak iterasi			
Durasi Pencarian	1.15 s	1.07 s	1.15 s
Banyak Restart	10	10	10
Banyak iterasi per restart	[7, 5, 4, 8, 5, 7, 6, 3, 4, 5, 4]	[7, 3, 4, 6, 5, 3, 6, 5, 5, 5, 7]	[5, 8, 5, 4, 4, 6, 7, 3, 6, 9, 5]
Maximum restart	10	10	10

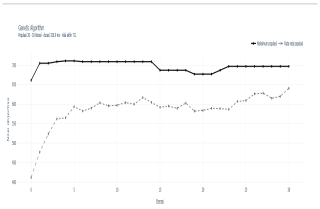
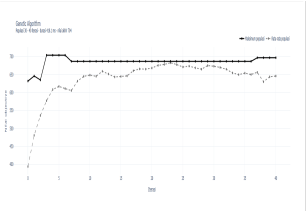
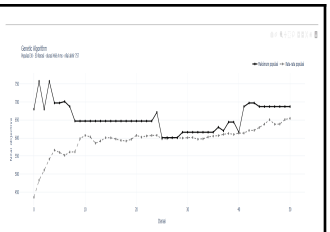
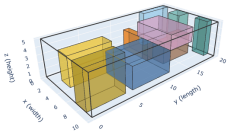
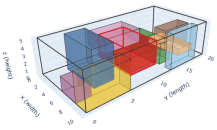
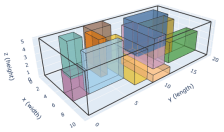
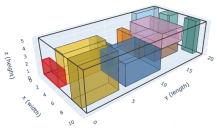
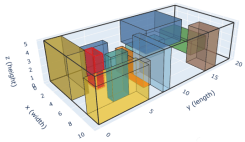
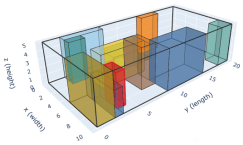
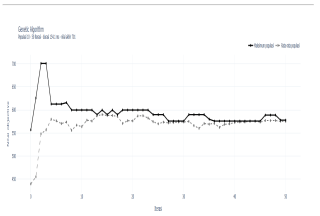
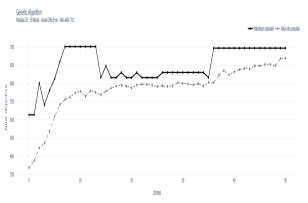
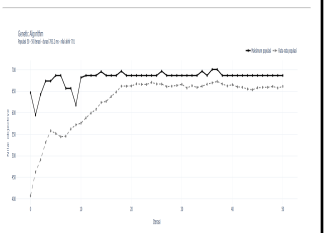
Simulated Annealing

Percobaan ke-i	**1**	**2**	**3**
State Awal			
State Akhir			
Nilai *Objective* Function akhir	757	757	711

Plot nilai *objective* terhadap banyak iterasi			
Durasi Pencarian	321.5 ms	290.0 ms	250.8 ms
Plot $e^{E/T}$ terhadap banyak iterasi yang telah dilewati			
Frekuensi *Stuck*	1319	1345	1308

Genetic Algorithm

Percobaan ke-i	**1**	**2**	**3**
Variable Control = Jumlah Populasi			
State Awal			
State Akhir			
Nilai *Objective* Function akhir	711	704	757

Plot nilai *objective* terhadap banyak iterasi			
Jumlah Populasi	30	30	30
Banyak Iterasi	30	40	50
Durasi Pencarian	326.9 ms	418.1 ms	468.4 ms
Variable Control = Banyak Iterasi			
State Awal			
State Akhir			
Nilai *Objective* Function akhir	701	701	701
Plot nilai *objective* terhadap banyak iterasi			
Jumlah Populasi	10	20	50
Banyak Iterasi	50	50	50
Durasi Pencarian	154.1 ms	296.8 ms	783.2 ms

2.5.2 Analisis Experiment

Hill-Climbing Steepest Ascent

Berdasarkan tiga percobaan, algoritma Hill-Climbing Steepest Ascent (HC-SA) menghasilkan nilai objektif akhir lebih dari 700. Algoritma dapat mencapai *global optimum* (757) pada percobaan ketiga jika dilihat dari batas maksimum total nilai paket berdasarkan kapasitas berat truk tanpa terkena penalti. Hill-Climbing Steepest Ascent hanya menerima *neighbor* dengan nilai yang lebih tinggi sehingga pencarian berhenti ketika tidak ditemukan *neighbor* yang lebih baik. Akibatnya, algoritma dapat berhenti pada optimum lokal

Jika berbicara mengenai durasi, *Steepest Ascent* lebih singkat dibanding algoritma-algoritma lainnya. Hal tersebut diakibatkan oleh iterasi yang dilewatkan oleh algoritmanya. *Steepest Ascent* hanya melakukan iterasi sebanyak 5 pada percobaan kesatu, 2 pada percobaan kedua, dan 6 pada percobaan ketiga. Berbeda jauh dengan *Stochastic Hill-Climbing* yang menjalankan 1000 iterasi sebelum berhenti. *Steepest Ascent* dapat selesai lebih cepat karena segera berhenti ketika tidak ada peningkatan. Karakteristik ini mendukung responsivitas, tetapi waktu pencarian yang singkat tidak selalu berarti solusi yang diperoleh sudah optimal.

Dari sisi kualitas hasil, *Steepest Ascent* menghasilkan rata-rata nilai objektif akhir sebesar 728, lebih tinggi dibandingkan *Stochastic Hill-Climbing* yang memperoleh sekitar 678,33. Namun, perbandingan ini terbatas pada percobaan yang dilakukan karena *state* awal kedua algoritma berbeda. Dari sisi konsistensi, nilai objektif akhir *Steepest Ascent* berada pada rentang 701–757, sehingga hasilnya masih bervariasi antarpercobaan. Ketiga grafik memang menunjukkan peningkatan nilai objektif, tetapi pola tersebut tergantung pada aturan algoritma yang hanya menerima *neighbor* dengan nilai lebih tinggi.

Hill-Climbing Sideways Move

Berdasarkan tiga percobaan, algoritma Hill-Climbing with Sideways Move menghasilkan nilai objektif akhir yang sama, yaitu 757. Dengan mengacu pada batas maksimum yang telah diperoleh, ketiga percobaan berhasil mencapai *global optimum*. Hasil ini lebih konsisten daripada algoritma Steepest Ascent. Kemampuan menerima *neighbor* dengan nilai objektif yang sama memungkinkan algoritma untuk *explore* lebih banyak susunan paket lain tanpa *stuck* di *local optimum* dengan puncak datar.

Grafik nilai objektif terlihat memiliki kenaikan dan bagian yang mendatar. Bagian mendatar menunjukkan perpindahan ke *state* (*neighbor*) dengan nilai objektif

yang sama. Kenaikan setelahnya menunjukkan bahwa algoritma dapat membuka solusi-solusi baru dibandingkan Steepest Ascent.

Dari sisi durasi, algoritma ini lebih lama dibandingkan algoritma lainnya. Hal tersebut dapat disebabkan oleh dilakukannya evaluasi seluruh *neighbor* yang dihasilkan pada setiap langkah serta pencarian yang tetap berlangsung ketika nilai objektifnya tidak meningkat.

Ketiga percobaan tersebut berhenti setelah mencapai batas 100 *sideways move*, dengan jumlah iterasi masing-masing 106, 107, dan 110. Hasil akhir yang sama menunjukkan konsistensi solusi pada ketiga percobaan. Dibandingkan Steepest Ascent, algoritma ini memberikan *searching* lebih luas dengan mengorbankan waktu durasi yang lebih lama.

Hill-Climbing Stochastic

Berdasarkan tiga percobaan, algoritma Stochastic Hill-Climbing menghasilkan nilai objektif akhir sebesar 625, 713, dan 697. Pemeriksaan terhadap seluruh *neighbor* yang dihasilkan oleh fungsi `get_all_neighbor()` pada ketiga *state* akhir tidak memiliki *neighbor* dengan nilai objektif lebih tinggi, meskipun masih terdapat *neighbor* yang bernilai sama. Karena algoritma hanya menerima nilai objektif yang lebih baik, *neighbor* bernilai sama tidak dapat digunakan.

Kondisi tersebut terjadi karena pemilihan *neighbor* secara acak tidak mengubah penerimaan yang hanya memperbolehkan *neighbor* dengan nilai objektif lebih tinggi. *Neighbor* dengan nilai yang sama atau lebih rendah akan ditolak menyebabkan adanya bagian pada grafik *objective plot* yang datar dengan arti *state* tidak berubah.

Dari sisi durasi, ketiga percobaan membutuhkan waktu dengan rata-rata sekitar 120.7 ms. Seluruh percobaan berhenti pada 1.000 iterasi sesuai dengan input `max_iteration`. Lebih singkat dibandingkan Sideways Move karena hanya mengevaluasi satu kandidat *neighbor* dan lebih lama dibandingkan Steepest Ascent. Dari sisi konsistensi solusi, nilai akhir Stochastic sangat tidak konsisten bahkan sangat bervariasi. Variasi ini dipengaruhi oleh *state* awal serta *neighbor* yang dipilih secara acak.

Hill-Climbing Random Restart

Berdasarkan hasil percobaan, algoritma Hill-Climbing Random Restart selalu menghasilkan nilai objektif sebesar 757, sehingga berhasil mencapai *global optimum*. Percobaan menggunakan batas sepuluh *restart* yang artinya terdapat sebelas proses pencarian. Algoritma akan menyimpan solusi terbaik dari seluruh proses tersebut. Algoritma ini dapat membantu menemukan solusi yang lebih baik ketika pencarian awal berhenti pada hasil yang belum optimal.

Random Restart tidak memiliki ketergantungan antara satu *state restart* dengan lainnya. Setiap *searching* masih dapat berhenti pada *local optimum*, tetapi *restart* memberikan kesempatan untuk *explore* lebih dari ruang pencarian. Grafik objektif menunjukkan bahwa setiap proses pencarian mengalami peningkatan tetapi tidak konsisten.

Dari sisi durasi, ketiga percobaan membutuhkan waktu yang lumayan lama dengan rata-rata 1,12 detik. Tambahan waktu diperlukan untuk membangkitkan *state* acak dan menjalankan pencarian berulang kali. Implementasi tetap menjalankan seluruh sepuluh *restart* meskipun *global optimum* sudah ditemukan karena penghentiannya ditentukan dari batas *restart* yang diizinkan.

Dari sisi konsistensi, ketiga percobaan menghasilkan solusi akhir yang sama, meskipun hasil setiap *restart* sangat bervariasi jika diacu pada grafik. Konsistensi ini lebih baik daripada algoritma Steepest Ascent dan Stochastic. Namun, keberhasilan tersebut juga sangat bergantung dengan jumlah maksimum *restart* yang dilakukan untuk eksplorasi.

Simulated Annealing

Dari hasil ketiga percobaan pada algoritma Simulated Annealing (SA) menghasilkan nilai objektif akhir lebih dari 700. Dua dari tiga percobaan tersebut mencapai nilai *global optimum* yaitu 757. Algoritma ini mampu mencapai *global optimum* dikarenakan menerima 'penurunan' nilai objektif, hal ini bertujuan agar algoritma ini menghindari hasil *local optimum*.

Berdasarkan hasil percobaan algoritma ini masih memiliki kecepatan yang cukup baik dibandingkan dengan algoritma lainnya. Algoritma ini mampu mendapatkan waktu durasi < 500 ms dibandingkan dengan algoritma lain yang bisa mencapai > 1 s algoritma ini masih tergolong cepat. Dengan kecepatan yang cukup cepat, algoritma ini juga memiliki akurasi nilai akhir yang serupa baik hingga mencapai *global optimum*. Oleh karena itu, algoritma dapat digolongkan konsisten berdasarkan hasil percobaan tersebut. Meskipun memiliki hasil konsisten algoritma ini masih sering '*stuck*' setiap iterasinya tidak terjadi perubahan nilai objektif value.

Dikarenakan pada implementasi algoritma Simulated Annealing (SA) menggunakan random generate threshold. *Neighbor* yang diterima apabila memiliki value objektif lebih kecil akan sangat memengaruhi hasil akhirnya. Penerimaan tersebut dapat memengaruhi hasil dikarenakan *neighbor* memiliki nilai kecil akan tetapi akhirnya algoritma tersebut tetap terjebak di dalam *local optimum* sehingga *state global optimum* tidak tercapai.

Genetic Algorithm

Berdasarkan hasil percobaan dengan jumlah populasi sebagai variabel kontrol pada algoritma *Genetic Algorithm* (GA), didapatkan nilai objektif akhir dari ketiga percobaan 711, 704, dan 757. Populasi dibuat tetap dengan jumlah 30 individu, sedangkan banyak iterasi divariasikan antara 30, 40, dan 50 iterasi (*generations)*. Dapat terlihat bahwa pada percobaan ketiga, percobaan mendapatkan nilai objektif akhir 757 yang merupakan nilai *global optimum*, sedangkan dua percobaan lainnya berhenti di *local optimum.*

Penambahan iterasi tidak menunjukkan pengaruh yang konsisten terhadap nilai objektif akhir, karena pada 30 iterasi mendapatkan hasil 711 sedangkan pada 40 iterasi mendapatkan hasil 704, namun jumlah iterasi menaikkan rata-rata populasi terutama pada iterasi terakhir. Sehingga populasi secara keseluruhan selalu mencoba dan bergerak mendapatkan hasil solusi yang lebih baik, walaupun tidak selalu menghasilkan individu baru yang lebih baik dari nilai sebelumnya. Kemudian durasi juga bertambah hampir linear karena banyak iterasi bertambah.

Pada percobaan dengan banyak iterasi sebagai variabel kontrol, didapatkan nilai objektif akhir dari ketiga percobaan berupa 701. Iterasi dibuat tetap sebanyak 50 iterasi, sedangkan jumlah populasi bervariasi antara 10, 20, dan 50 individu. Dapat terlihat bahwa nilai objektif dari ketiga percobaan mendapatkan hasil yang sama, yaitu 701 yang merupakan *local optimum*. Hal ini bisa terjadi karena setiap kali *genetic algorithm* dijalankan akan inisialisasi populasi awal secara *random* sehingga setiap percobaan yang dilakukan bisa mendapatkan hasil berbeda-beda. Jumlah populasi yang lebih besar berarti lebih banyak individu *random* di awal, sehingga rata-rata nilai objektif akhir seharusnya lebih besar dibandingkan jumlah populasi yang lebih kecil sehingga membuatnya lebih konsisten.

Durasi juga ikut bertambah secara linear terhadap jumlah populasi karena semakin besar jumlah populasi waktu yang diperlukan untuk setiap iterasi melewati fungsi `genetic_algorithm` juga ikut bertambah, karena harus melalui proses *fitness, fitness_percentage, selection, crossover* dan *repair, serta mutasi.*

Konsistensi dari ketiga percobaan dapat terlihat mendapatkan nilai objektif akhir yang sama yaitu 701, meskipun nilai objektif akhir dari setiap percobaan dapat bergantung dengan berapa kali percobaan tersebut dilakukan. Hal ini dikarenakan populasi awal, *selection roulette wheel, crossover,* dan *mutation* semuanya dilakukan secara *random.*

BAB III

KESIMPULAN DAN SARAN

3.1 Kesimpulan

Berdasarkan seluruh hasil experiment yang telah dilakukan, seluruh algoritma *local search* yang dilakukan dapat menghasilkan *state* akhir dari *package* dengan tepat dan mendekati *global optimum.* Algoritma Hill-Climbing Steepest Ascent dan Hill-Climbing Stochastic hanya menerima *neighbor* yang nilainya lebih baik dibandingkan *current state,* namun Stochastic melakukan pemilihan *neighbor* secara *random.* Sideways Move dan Random Restart membutuhkan waktu eksekusi paling lama karena mencoba lebih banyak *state* dibandingkan algoritma yang lain. Algoritma Simulated Annealing sendiri karena dapat menerima *neighbor state* yang nilainya lebih buruk dibandingkan *current state,* maka algoritma ini memberikan keseimbangan terbaik antara kualitas dan waktu eksekusi dan mampu mengatasi *local optimum* dengan baik.

Algoritma Genetic Algorithm dilakukan dengan berbagai tahap, mulai dari inisialisasi populasi secara *random*, *selection, crossover,* dan *mutation*. Jumlah populasi pada algoritma *genetic algorithm* lebih memengaruhi hasil dibandingkan variabel banyak iterasi, meskipun keduanya tetap meningkatkan durasi eksekusi dari percobaan yang dilakukan. Kemampuan algoritma untuk keluar dari *local optimum* menggunakan algoritma masing-masing lebih menentukan nilai *objective* akhir lebih memengaruhi hasil dibandingkan waktu eksekusi dari setiap percobaan.

3.2 Saran

Berdasarkan hasil yang telah diperoleh, algoritma *local search* yang sudah diimplementasikan masih dapat lebih ditingkatkan. Salah satu saran yang dapat diberikan adalah meningkatkan *objective function* yang didefinisikan dengan menggunakan parameter-parameter lainnya. Selain itu, eksperimen sebaiknya dapat diuji dengan test case dengan jumlah *package* yang lebih besar dan beragam, sehingga perbandingan antara algoritma dapat menjadi lebih akurat.

PEMBAGIAN TUGAS

NIM	NAMA	PEMBAGIAN TUGAS
13524005	Geraldo Artemius	Algoritma Hill Climbing, Input, Output, Objective Function, Laporan
13524055	Junior Natra Situmorang	Algoritma Simulated Annealing, Objective Function, Laporan
13524103	Reynard Nathanael	Genetic Algorithm, Models, Visualizer, Laporan

REFERENSI

[1] J. Brownlee, “Simulated Annealing Afternoon,” Algorithmafternoon.com. Accessed: Oct. 19, 2026. [Online]. Available: https://algorithmafternoon.com/books/simulated_annealing/

[2] J. Blank, “Repair — Pymoo: Multi-Objective Optimization in Python 0.6.2 Documentation,” Pymoo.org. Accessed: Oct. 19, 2026. [Online]. Available: https://pymoo.org/operators/repair.html

[3] plotlygraphs, “Line Charts,” plotly.com. Accessed: Oct. 19, 2026. [Online]. Available: https://plotly.com/python/line-charts/

[4] “3D,” plotly.com. Accessed: Oct. 19, 2026. [Online]. Available: https://plotly.com/python/3d-mesh/

[5] “Beyond Classical Search,” Edunex, 2026. Accessed: Oct. 20, 2026. [Online]. Available: https://edunexcontentprodhot.blob.core.windows.net/edunex/2026/97453-Artificial-Intelligence-Parent-Class/461472-Beyond-Classical-Search/171817-Local-Search/file/1788061222110_2026_IF3170_Materi02-BeyondClassicialSearch?sv=2024-11-04&spr=https&st=2026-08-30T03%3A40%3A21Z&se=2028-08-29T03%3A40%3A21Z&sr=b&sp=r&sig=lsfDzNIMuPcnM9TNeoawD13JwzN6HCd7iKrvYlhKORU%3D&rsct=application%2Fpdf.

[6] “Python 3.15.0 documentation,” docs.python.org. Accessed: Oct. 20, 2026. [Online]. Available: https://docs.python.org/3/.

LAMPIRAN FORM PENGGUNAAN AI

FORMAT DEKLARASI PENGGUNAANAI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami: : Geraldo Artemius (13524005), Junior Natra Situmorang (13524055), Reynard Nathanael (13524103)
Nama/NIM
Fakultas/Sekolah : STEI-K/Teknik Informatika
Kode Mata Kuliah : IF3170
Nama Mata Kuliah : Inteligensi Artifisial
Judul Tugas/Proposal : Tugas Besar 1 Inteligensi Artifisial

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Tidak	
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	AI-assisted workflow
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	
Lainnya, Jelaskan:		

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 20 Oktober 2026



Geraldo Artemius
13524005



Junior Natra Situmorang
13524055



Reynard Nathanael
13524103